PYTHON PROGRAMMING LAB RECORD

Experiments compiled from the 10 uploaded Python source files

Student Name: _____

Register Number: _____

Department / Class: _____

Academic Year: _____

Prepared for academic practical record submission

TABLE OF CONTENTS

Experiment 1: Employee Payroll and Deduction Analysis 4

Experiment 2: Campus Event Financial Planning 6

Experiment 3: Campus Event Financial Planning (Duplicate Upload) 8

Experiment 4: ATM Transaction Sequence Simulation 10

Experiment 5: Attendance Recovery Computation 12

Experiment 6: Smart Parking Fee Management 14

Experiment 7: Student Feedback Sentiment Scoring 16

Experiment 8: Student Marks Matrix Analysis 18

Experiment 9: Student Marks Matrix Analysis (Duplicate Upload) 20

Experiment 10: University Scholarship Allocation 22

Experiment 1: Employee Payroll and Deduction Analysis

1. Aim

Calculate gross salary from basic pay, allowances, overtime and incentives, apply progressive deductions, and print a salary statement.

2. Procedure

1. Open the supplied .py file in a Python 3 editor or IDE.

2. Read the input prompts and prepare the sample values shown in this record.

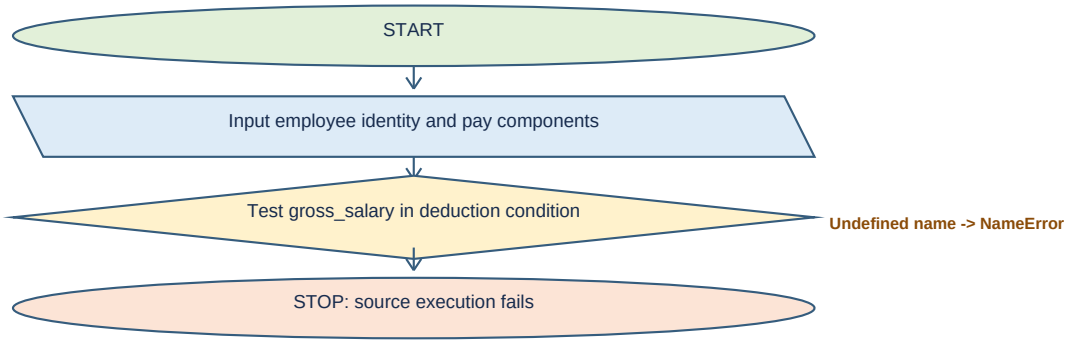
3. Run the original file without changing its contents, then enter the values in the requested order.

4. Observe the program output or Python exception and compare it with the expected behavior.

5. Record any syntax, name, or runtime error; do not report an unsuccessful run as a successful execution.

3. Flowchart (standard symbols)

Oval: Start/Stop Parallelogram: Input/Output Rectangle: Process Diamond: Decision



4. Pseudocode

```
BEGIN
INPUT employee name, ID, basic pay, HRA, DA, overtime hours, overtime rate and incentive
IF gross_salary <= 10000 THEN deduction = 0
ELSE IF gross_salary <= 20000 THEN deduction = 5% of amount above 10000
ELSE deduction = 500 + 10% of amount above 20000
OUTPUT earnings, deduction and net salary
END
```

5. Algorithm

1. START

2. Read employee name, ID, basic salary, HRA, DA, overtime hours, overtime rate and incentive.

3. The source tests gross_salary in an IF statement before assigning it.

4. Python raises NameError; deduction and salary-slip calculations are not reached.

5. STOP.

6. Source Code (original upload; unchanged)

```
name = input("Enter Employee Name: ")
emp_id = input("Enter Employee ID: ")

basic = float(input("Enter Basic Salary: "))
hra = float(input("Enter House Rent Allowance: "))
da = float(input("Enter Dearness Allowance: "))
overtime_hours = float(input("Enter Overtime Hours: "))
overtime_rate = float(input("Enter Overtime Rate per Hour: "))
incentive = float(input("Enter Incentive Amount: "))
if gross_salary <= 10000:
    deduction = 0

elif gross_salary <= 20000:
    deduction = (gross_salary - 10000) * 0.05

else:
    deduction = (10000 * 0.05) + ((gross_salary - 20000) * 0.10)
```

```
print("\n========================================")
print("          EMPLOYEE SALARY SLIP")
print("========================================")

print("Employee Name       :", name)
print("Employee ID         :", emp_id)

print("----------------------------------------")
print("              EARNINGS")
print("----------------------------------------")

print("Basic Salary        : Rs.", basic)
print("House Rent Allowance: Rs.", hra)
print("Dearness Allowance  : Rs.", da)
print("Overtime Amount     : Rs.", overtime)
print("Incentive Amount    : Rs.", incentive)

print("----------------------------------------")
print("Gross Salary        : Rs.", round(gross_salary, 2))

print("----------------------------------------")
print("              DEDUCTIONS")
print("----------------------------------------")

print("Total Deduction     : Rs.", round(deduction, 2))

print("----------------------------------------")
print("Net Salary          : Rs.", round(net_salary, 2))
print("========================================")
print("       Salary Process Completed")
print("========================================")
```

7. Output (actual execution)

Sample input supplied to the original file:

```
Rahul
E101
20000
3000
2000
10
100
2000
```

Captured output / error:

```
Enter Employee Name: Enter Employee ID: Enter Basic Salary: Enter House Rent Allowance: Enter Dearness
Allowance: Enter Overtime Hours: Enter Overtime Rate per Hour: Enter Incentive Amount: Traceback (most recent
call last):
  File ■[35m"/mnt/data/qw.py"■[0m, line ■[35m11■[0m, in ■[35m<module>■[0m
    if ■[1;31mgross_salary■[0m <= 10000:
       ■[1;31m^^^^^^^^^^^■[0m
■[1;35mNameError■[0m: ■[35mname 'gross_salary' is not defined■[0m
```

Execution status: Did not complete successfully (exit code 1). Output is from execution of the original uploaded file.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01: Typical salary	Rahul | E101 | 20000 | 3000 | 2000 | 10 | 100 | 2000	Salary slip is printed	■[1;31m^^^^^^^^^^■[0m | ■[1;35mNameError■[0m: ■[35mname 'gross_salary' is not defined■[0m	FAIL
TC02: Lower salary boundary	A | E1 | 8000 | 500 | 500 | 0 | 100 | 0	Salary slip is printed	■[1;31m^^^^^^^^^^■[0m | ■[1;35mNameError■[0m: ■[35mname 'gross_salary' is not defined■[0m	FAIL
TC03: Higher salary boundary	B | E2 | 30000 | 2000 | 1000 | 5 | 100 | 1000	Salary slip is printed	■[1;31m^^^^^^^^^^■[0m | ■[1;35mNameError■[0m: ■[35mname 'gross_salary' is not defined■[0m	FAIL

9. Result

The uploaded payroll source accepts employee details but does not calculate gross_salary, overtime or net_salary before referencing them. Execution fails with NameError; successful payroll generation is not verified.

Experiment 2: Campus Event Financial Planning

1. Aim

Calculate fixed and variable event costs, sponsorship shortfall, and the registration count needed to cover the shortfall.

2. Procedure

1. Open the supplied .py file in a Python 3 editor or IDE.

2. Read the input prompts and prepare the sample values shown in this record.

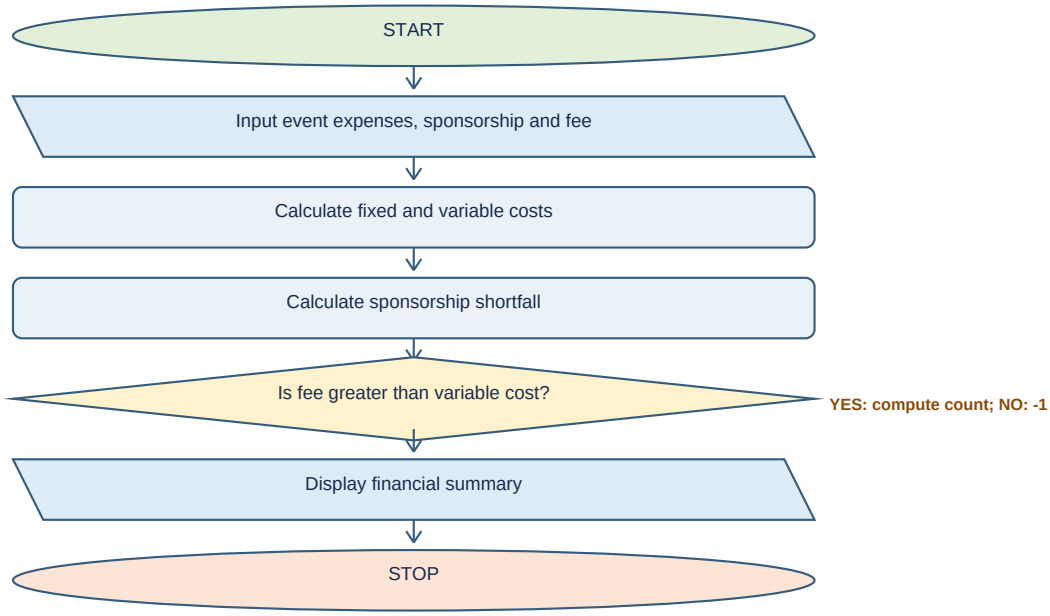
3. Run the original file without changing its contents, then enter the values in the requested order.

4. Observe the program output or Python exception and compare it with the expected behavior.

5. Record any syntax, name, or runtime error; do not report an unsuccessful run as a successful execution.

3. Flowchart (standard symbols)

Oval: Start/Stop Parallelogram: Input/Output Rectangle: Process Diamond: Decision



4. Pseudocode

```
BEGIN
INPUT venue, equipment, catering/person, transport, certificate/person, sponsorship and registration fee
fixed = venue + equipment + transport
variable = catering + certificate
shortfall = MAX(0, fixed - sponsorship)
IF fee > variable AND shortfall > 0 THEN calculate break_even
ELSE IF shortfall = 0 THEN break_even = 0 ELSE break_even = -1
OUTPUT fixed cost, variable cost, shortfall and break_even
END
```

5. Algorithm

1. START

2. Read venue, equipment, catering/person, transport, certificate/person, sponsorship and registration fee.

3. Calculate fixed cost = venue + equipment + transport and variable cost/person = catering + certificate.

4. Calculate sponsorship shortfall = max(0, fixed cost - sponsorship).

5. If fee > variable cost and shortfall > 0, calculate the break-even count; if shortfall is zero set count to zero; otherwise set it to -1.

6. Print the fixed cost, variable cost, shortfall and participant count.

7. STOP.

6. Source Code (original upload; unchanged)

```
venue = float(input("Enter venue cost: "))
equipment = float(input("Enter equipment cost: "))
catering = float(input("Enter catering cost per person: "))
transport = float(input("Enter transport cost: "))
certificate = float(input("Enter certificate cost per person: "))
```

```
sponsorship = float(input("Enter sponsorship amount: "))
fee = float(input("Enter registration fee per participant: "))
fixed = venue + equipment + transport
variable = catering + certificate
shortfall = max(0, fixed - sponsorship)
break_even = int(shortfall / (fee - variable)) + 1 if fee > variable and shortfall > 0 else 0 if shortfall ==
0 else -1
print("Total fixed cost:", fixed, "\nVariable cost/person:", variable, "\nSponsorship shortfall:", shortfall,
"\nMinimum participants:", break_even)
```

7. Output (actual execution)

Sample input supplied to the original file:

```
50000
10000
200
5000
50
20000
500
```

Captured output / error:

```
Enter venue cost: Enter equipment cost: Enter catering cost per person: Enter transport cost: Enter
certificate cost per person: Enter sponsorship amount: Enter registration fee per participant: Total fixed
cost: 65000.0
Variable cost/person: 250.0
Sponsorship shortfall: 45000.0
Minimum participants: 181
```

Execution status: Completed with exit code 0. Output is from execution of the original uploaded file.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01: Typical event	50000 | 10000 | 200 | 5000 | 50 | 20000 | 500	Minimum participants: 181	| Variable cost/person: 250.0 | Sponsorship shortfall: 45000.0 | Minimum participants: 181	PASS
TC02: Sponsorship covers fixed costs	10000 | 2000 | 100 | 1000 | 20 | 20000 | 200	Minimum participants: 0	3000.0 | Variable cost/person: 120.0 | Sponsorship shortfall: 0 | Minimum participants: 0	PASS
TC03: Registration fee below variable cost	10000 | 2000 | 300 | 1000 | 50 | 0 | 200	Minimum participants: -1	| Variable cost/person: 350.0 | Sponsorship shortfall: 13000.0 | Minimum participants: -1	PASS

9. Result

The uploaded event-planning program executed successfully for the tested cases and reported fixed cost, variable cost, sponsorship shortfall and break-even estimate.

Experiment 3: Campus Event Financial Planning (Duplicate Upload)

1. Aim

Analyse the uploaded duplicate event-planning program using the source exactly as supplied.

2. Procedure

1. Open the supplied .py file in a Python 3 editor or IDE.

2. Read the input prompts and prepare the sample values shown in this record.

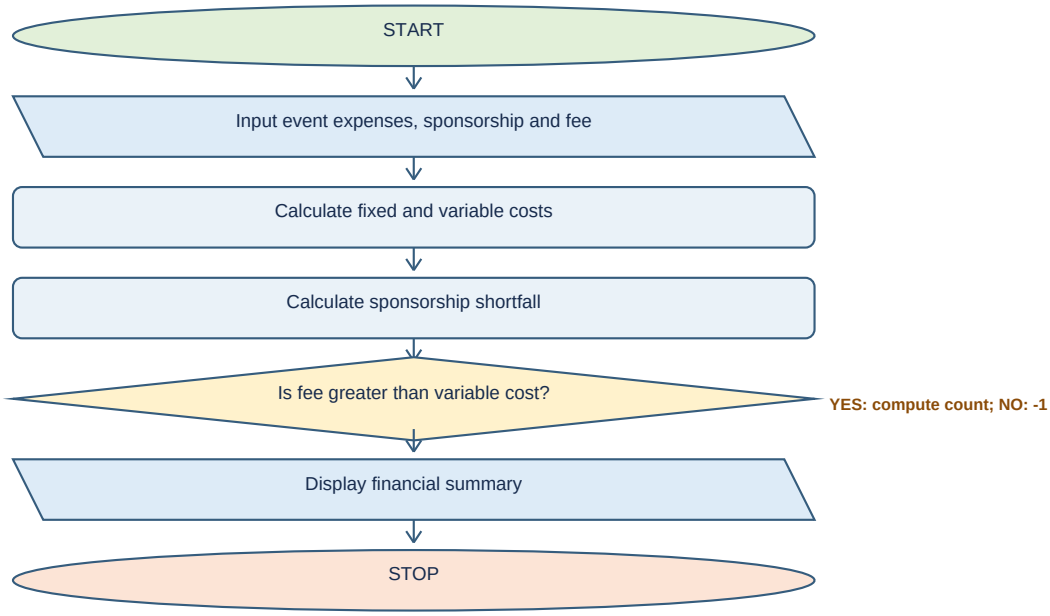
3. Run the original file without changing its contents, then enter the values in the requested order.

4. Observe the program output or Python exception and compare it with the expected behavior.

5. Record any syntax, name, or runtime error; do not report an unsuccessful run as a successful execution.

3. Flowchart (standard symbols)

Oval: Start/Stop Parallelogram: Input/Output Rectangle: Process Diamond: Decision



4. Pseudocode

```
BEGIN
INPUT event expenses, sponsorship and registration fee
CALCULATE fixed = venue + equipment + transport
CALCULATE variable = catering + certificate
CALCULATE shortfall = MAX(0, fixed - sponsorship)
IF fee > variable AND shortfall > 0 THEN calculate break_even
ELSE IF shortfall = 0 THEN break_even = 0 ELSE break_even = -1
OUTPUT financial summary
END
```

5. Algorithm

1. START

2. Read event costs, sponsorship and registration fee.

3. Calculate fixed and per-person variable costs.

4. Calculate the sponsorship shortfall.

5. Apply the conditional expression to calculate break-even count, set zero when there is no shortfall, or set -1 when fee does not exceed variable cost.

6. Print the financial summary.

7. STOP.

6. Source Code (original upload; unchanged)

```
venue = float(input("Enter venue cost: "))
equipment = float(input("Enter equipment cost: "))
catering = float(input("Enter catering cost per person: "))
transport = float(input("Enter transport cost: "))
```

```
certificate = float(input("Enter certificate cost per person: "))
sponsorship = float(input("Enter sponsorship amount: "))
fee = float(input("Enter registration fee per participant: "))
fixed = venue + equipment + transport
variable = catering + certificate
shortfall = max(0, fixed - sponsorship)
break_even = int(shortfall / (fee - variable)) + 1 if fee > variable and shortfall > 0 else 0 if shortfall ==
0 else -1
print("Total fixed cost:", fixed, "\nVariable cost/person:", variable, "\nSponsorship shortfall:", shortfall,
"\nMinimum participants:", break_even)
```

7. Output (actual execution)

Sample input supplied to the original file:

```
50000
10000
200
5000
50
20000
500
```

Captured output / error:

```
Enter venue cost: Enter equipment cost: Enter catering cost per person: Enter transport cost: Enter
certificate cost per person: Enter sponsorship amount: Enter registration fee per participant: Total fixed
cost: 65000.0
Variable cost/person: 250.0
Sponsorship shortfall: 45000.0
Minimum participants: 181
```

Execution status: Completed with exit code 0. Output is from execution of the original uploaded file.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01: Typical event	50000 | 10000 | 200 | 5000 | 50 | 20000 | 500	Minimum participants: 181	| Variable cost/person: 250.0 | Sponsorship shortfall: 45000.0 | Minimum participants: 181	PASS
TC02: Sponsorship covers fixed costs	10000 | 2000 | 100 | 1000 | 20 | 20000 | 200	Minimum participants: 0	3000.0 | Variable cost/person: 120.0 | Sponsorship shortfall: 0 | Minimum participants: 0	PASS
TC03: Registration fee below variable cost	10000 | 2000 | 300 | 1000 | 50 | 0 | 200	Minimum participants: -1	| Variable cost/person: 350.0 | Sponsorship shortfall: 13000.0 | Minimum participants: -1	PASS

9. Result

The uploaded event-planning program executed successfully for the tested cases and reported fixed cost, variable cost, sponsorship shortfall and break-even estimate.

Experiment 4: ATM Transaction Sequence Simulation

1. Aim

Process deposits and withdrawals, validate transaction amounts, enforce a per-transaction withdrawal limit, and report the balance.

2. Procedure

1. Open the supplied .py file in a Python 3 editor or IDE.

2. Read the input prompts and prepare the sample values shown in this record.

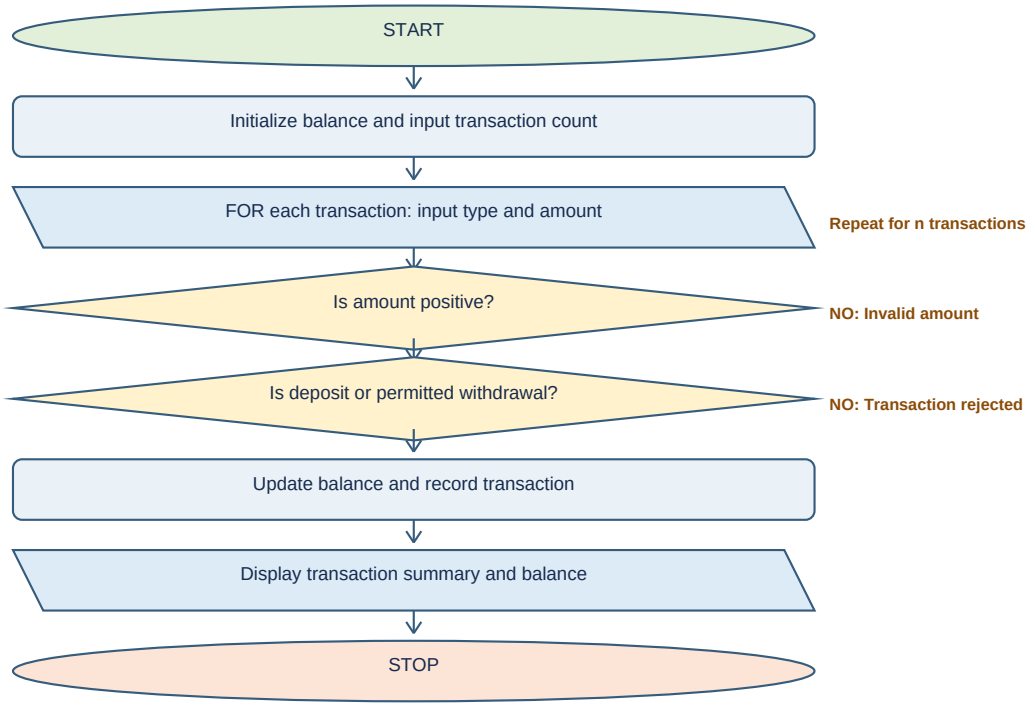
3. Run the original file without changing its contents, then enter the values in the requested order.

4. Observe the program output or Python exception and compare it with the expected behavior.

5. Record any syntax, name, or runtime error; do not report an unsuccessful run as a successful execution.

3. Flowchart (standard symbols)

Oval: Start/Stop Parallelogram: Input/Output Rectangle: Process Diamond: Decision



4. Pseudocode

```
BEGIN
SET balance = 5000; SET transactions = empty list
INPUT number of transactions
FOR each transaction: INPUT type and amount
IF amount <= 0 THEN OUTPUT Invalid amount
ELSE IF type is deposit THEN add amount to balance and record deposit
ELSE IF type is withdrawal AND amount <= balance AND amount <= 20000 THEN subtract and record withdrawal
ELSE OUTPUT Transaction rejected
OUTPUT transaction summary and final balance
END
```

5. Algorithm

1. START

2. Initialize balance to 5000 and create an empty transaction list.

3. Read the number of transactions.

4. FOR each transaction, read its type and amount.

5. If amount <= 0, print Invalid amount. If deposit, add amount to balance and record it.

6. If withdrawal is within balance and 20000 limit, subtract and record it; otherwise reject the transaction.

7. Print transaction list and final balance.

8. STOP.

6. Source Code (original upload; unchanged)

```
balance = 5000
transactions = []
n = int(input("Enter number of transactions: "))
for i in range(n):
    choice = input("Enter deposit or withdraw: ").lower()
    amount = float(input("Enter amount: "))
    if amount <= 0: print("Invalid amount")
    elif choice == "deposit": balance += amount; transactions.append(("Deposit", amount))
    elif choice == "withdraw" and amount <= balance and amount <= 20000: balance -= amount; transactions.
append(("Withdrawal", amount))
    else: print("Transaction rejected")
print("\nTransaction Summary:", transactions, "\nFinal Balance: Rs.", balance)
```

7. Output (actual execution)

Sample input supplied to the original file:

```
3
deposit
2000
withdraw
1000
withdraw
8000
```

Captured output / error:

```
Enter number of transactions: Enter deposit or withdraw: Enter amount: Enter deposit or withdraw: Enter
amount: Enter deposit or withdraw: Enter amount: Transaction rejected

Transaction Summary: [('Deposit', 2000.0), ('Withdrawal', 1000.0)]
Final Balance: Rs. 6000.0
```

Execution status: Completed with exit code 0. Output is from execution of the original uploaded file.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01: Valid deposit and withdrawal	3 | deposit | 2000 | withdraw | 1000 | withdraw | 8000	Final Balance: Rs. 6000.0	nsaction Summary: [('Deposit', 2000.0), ('Withdrawal', 1000.0)] | Final Balance: Rs. 6000.0	PASS
TC02: Withdrawal exceeds balance	1 | withdraw | 6000	Transaction rejected	: Enter amount: Transaction rejected | | Transaction Summary: [] | Final Balance: Rs. 5000	PASS
TC03: Invalid zero amount	1 | deposit | 0	Invalid amount	thdraw: Enter amount: Invalid amount | | Transaction Summary: [] | Final Balance: Rs. 5000	PASS

9. Result

The uploaded ATM simulation executed successfully for the tested deposit, withdrawal, insufficient-balance and invalid-amount cases.

Experiment 5: Attendance Recovery Computation

1. Aim

Calculate current attendance and determine the minimum number of consecutive attended classes needed to reach a target percentage.

2. Procedure

1. Open the supplied .py file in a Python 3 editor or IDE.

2. Read the input prompts and prepare the sample values shown in this record.

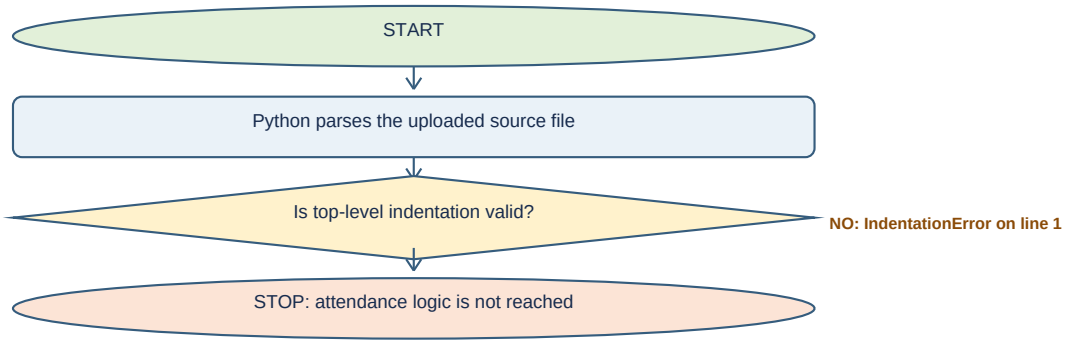
3. Run the original file without changing its contents, then enter the values in the requested order.

4. Observe the program output or Python exception and compare it with the expected behavior.

5. Record any syntax, name, or runtime error; do not report an unsuccessful run as a successful execution.

3. Flowchart (standard symbols)

Oval: Start/Stop Parallelogram: Input/Output Rectangle: Process Diamond: Decision



4. Pseudocode

```
BEGIN
Attempt to parse the uploaded Python file as written
The first top-level assignment has an unexpected leading indentation
Python raises IndentationError before any INPUT or attendance calculation
END
```

5. Algorithm

1. START

2. Attempt to execute the uploaded source file exactly as supplied.

3. Python encounters one leading space before the first top-level assignment and raises IndentationError.

4. No attendance percentage or recovery calculation is executed.

5. STOP.

6. Source Code (original upload; unchanged)

```
 total = int(input("Enter total classes conducted: "))
attended = int(input("Enter classes attended: "))
threshold = float(input("Enter eligibility percentage: "))
current = (attended / total) * 100
print("Current Attendance:", round(current, 2), "%")
if current >= threshold:
    print("You are already eligible")
else:
    required = 0
    while ((attended + required) / (total + required)) * 100 < threshold: required += 1
    print("Minimum consecutive classes required:", required)
```

7. Output (actual execution)

Sample input supplied to the original file:

```
100
65
75
```

Captured output / error:

```
File ■[35m"/mnt/data/me.py"■[0m, line ■[35m1■[0m
    total = int(input("Enter total classes conducted: "))
■[1;35mIndentationError■[0m: ■[35munexpected indent■[0m
```

Execution status: Did not complete successfully (exit code 1). Output is from execution of the original uploaded file.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01: Below threshold	100 | 65 | 75	Minimum consecutive classes required: 40	Enter total classes conducted: ")) | ■[1;35mIndentationError■[0m: ■[35munexpected indent■[0m	FAIL
TC02: Already eligible	100 | 80 | 75	You are already eligible	Enter total classes conducted: ")) | ■[1;35mIndentationError■[0m: ■[35munexpected indent■[0m	FAIL
TC03: At threshold	100 | 75 | 75	You are already eligible	Enter total classes conducted: ")) | ■[1;35mIndentationError■[0m: ■[35munexpected indent■[0m	FAIL

9. Result

The uploaded file contains unexpected leading indentation on its first top-level statement. Python raises IndentationError, so attendance recovery is not executed.

Experiment 6: Smart Parking Fee Management

1. Aim

Calculate parking duration and fee based on vehicle type, overnight duration, and a daily fee cap.

2. Procedure

1. Open the supplied .py file in a Python 3 editor or IDE.

2. Read the input prompts and prepare the sample values shown in this record.

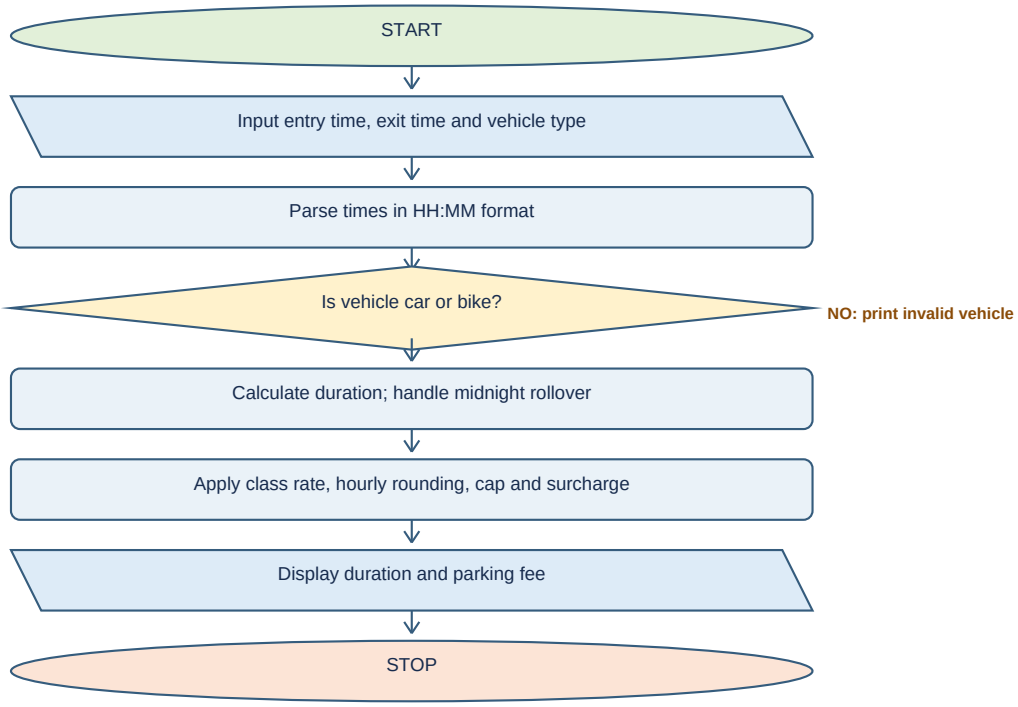
3. Run the original file without changing its contents, then enter the values in the requested order.

4. Observe the program output or Python exception and compare it with the expected behavior.

5. Record any syntax, name, or runtime error; do not report an unsuccessful run as a successful execution.

3. Flowchart (standard symbols)

Oval: Start/Stop Parallelogram: Input/Output Rectangle: Process Diamond: Decision



4. Pseudocode

```
BEGIN
INPUT entry time, exit time in HH:MM format, and vehicle type
PARSE entry and exit time using datetime.strptime
IF vehicle is not car or bike THEN OUTPUT invalid vehicle type
ELSE calculate elapsed hours; if hours <= 0, add 24 for overnight duration
SET rate = 30 for car OR 15 for bike
CALCULATE fee using rounded-up hours, 500 cap and 100 surcharge if hours > 12
OUTPUT parking hours and fee
END
```

5. Algorithm

1. START

2. Read entry time, exit time and vehicle type.

3. Parse times using the HH:MM format; malformed times raise a parsing error.

4. If vehicle is neither car nor bike, print Invalid vehicle type.

5. Otherwise calculate elapsed hours; if hours <= 0, add 24 to represent overnight parking.

6. Select rate 30 for car or 15 for bike. Calculate rounded-up hourly charge, apply the 500 cap and add 100 when duration exceeds 12 hours.

7. Print parking hours and fee.

8. STOP.

6. Source Code (original upload; unchanged)

```
from datetime import datetime
entry = datetime.strptime(input("Enter entry time (HH:MM): "), "%H:%M")
exit = datetime.strptime(input("Enter exit time (HH:MM): "), "%H:%M")
vehicle = input("Enter vehicle type (car/bike): ").lower()
if vehicle not in ["car", "bike"]: print("Invalid vehicle type")
else:
    hours = (exit - entry).total_seconds() / 3600
    hours = hours if hours > 0 else hours + 24
    rate = 30 if vehicle == "car" else 15
    fee = min(500, max(1, int(hours + 0.99)) * rate) + (100 if hours > 12 else 0)
    print("Parking hours:", round(hours, 2), "\nTotal parking fee: Rs.", fee if hours > 0 else "Invalid time")
```

7. Output (actual execution)

Sample input supplied to the original file:

```
09:00
13:00
car
```

Captured output / error:

```
Enter entry time (HH:MM): Enter exit time (HH:MM): Enter vehicle type (car/bike): Parking hours: 4.0
Total parking fee: Rs. 120
```

Execution status: Completed with exit code 0. Output is from execution of the original uploaded file.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01: Four-hour car parking	09:00 | 13:00 | car	Total parking fee: Rs. 120	ime (HH:MM): Enter vehicle type (car/bike): Parking hours: 4.0 | Total parking fee: Rs. 120	PASS
TC02: Overnight duration	20:00 | 08:00 | bike	Parking hours: 12.0	me (HH:MM): Enter vehicle type (car/bike): Parking hours: 12.0 | Total parking fee: Rs. 180	PASS
TC03: Invalid vehicle category	09:00 | 13:00 | bus	Invalid vehicle type	y time (HH:MM): Enter exit time (HH:MM): Enter vehicle type (car/bike): Invalid vehicle type	PASS

9. Result

The uploaded parking program executed successfully for the tested car, overnight-bike and invalid-vehicle cases.

Experiment 7: Student Feedback Sentiment Scoring

1. Aim

Count positive and negative keyword indicators in multiple feedback sentences and classify each sentiment.

2. Procedure

1. Open the supplied .py file in a Python 3 editor or IDE.

2. Read the input prompts and prepare the sample values shown in this record.

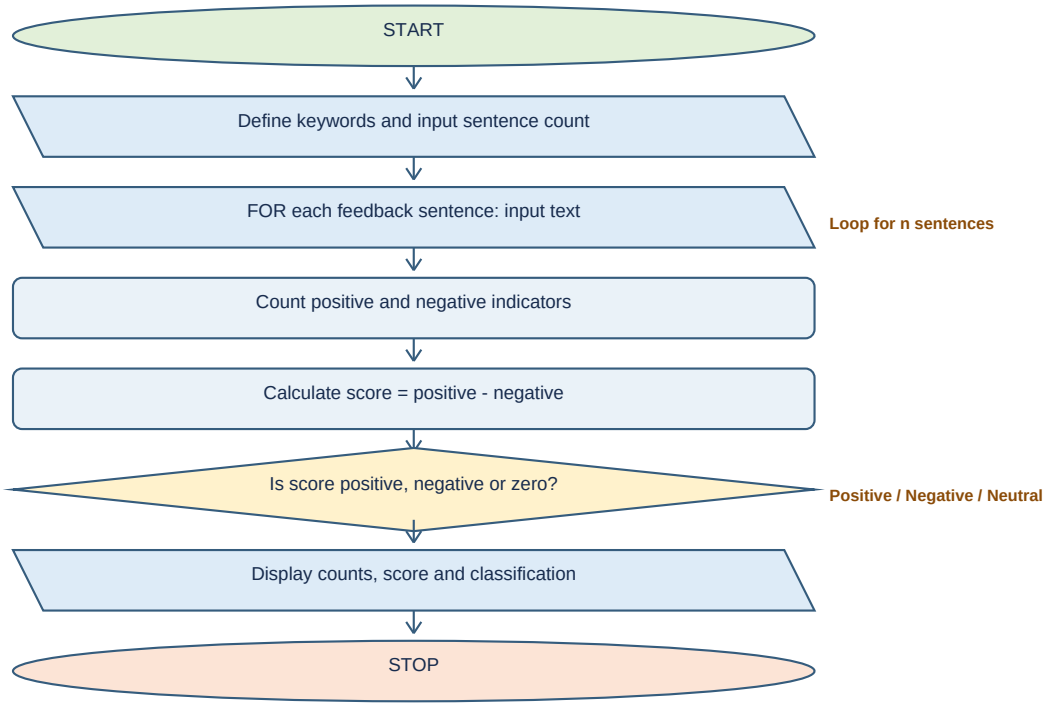
3. Run the original file without changing its contents, then enter the values in the requested order.

4. Observe the program output or Python exception and compare it with the expected behavior.

5. Record any syntax, name, or runtime error; do not report an unsuccessful run as a successful execution.

3. Flowchart (standard symbols)

Oval: Start/Stop Parallelogram: Input/Output Rectangle: Process Diamond: Decision



4. Pseudocode

```
BEGIN
SET positive and negative keyword lists
INPUT number of feedback sentences
FOR each feedback: INPUT text and convert to lowercase
COUNT positive and negative keyword matches
score = positive count - negative count
IF score > 0 THEN Positive; ELSE IF score < 0 THEN Negative; ELSE Neutral
OUTPUT counts, score and classification
END
```

5. Algorithm

1. START

2. Create positive and negative keyword lists and read the number of feedback sentences.

3. FOR each sentence, read feedback and convert it to lowercase.

4. Count positive and negative keywords found as substrings.

5. Calculate score = positive count - negative count.

6. Classify as Positive if score > 0, Negative if score < 0, otherwise Neutral.

7. Print counts, score and classification.

8. STOP.

6. Source Code (original upload; unchanged)

```
positive = ["good", "excellent", "helpful", "amazing", "nice"]
negative = ["bad", "poor", "boring", "worst", "difficult"]
n = int(input("Enter number of feedback sentences: "))
for i in range(n):
    feedback = input("Enter feedback: ").lower()
    pos = sum(word in feedback for word in positive)
    neg = sum(word in feedback for word in negative)
    score = pos - neg
    result = "Positive" if score > 0 else "Negative" if score < 0 else "Neutral"
    print("Positive:", pos, "Negative:", neg, "Score:", score)
    print("Feedback Classification:", result)
```

7. Output (actual execution)

Sample input supplied to the original file:

```
2
Excellent teaching and helpful staff
Poor teaching and boring classes
```

Captured output / error:

```
Enter number of feedback sentences: Enter feedback: Positive: 2 Negative: 0 Score: 2
Feedback Classification: Positive
Enter feedback: Positive: 0 Negative: 2 Score: -2
Feedback Classification: Negative
```

Execution status: Completed with exit code 0. Output is from execution of the original uploaded file.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01: Positive feedback	1 | Excellent teaching and helpful staff	Feedback Classification: Positive	tences: Enter feedback: Positive: 2 Negative: 0 Score: 2 | Feedback Classification: Positive	PASS
TC02: Negative feedback	1 | Poor teaching and boring classes	Feedback Classification: Negative	ences: Enter feedback: Positive: 0 Negative: 2 Score: -2 | Feedback Classification: Negative	PASS
TC03: Neutral feedback	1 | The class was held today	Feedback Classification: Neutral	ntences: Enter feedback: Positive: 0 Negative: 0 Score: 0 | Feedback Classification: Neutral	PASS

9. Result

The uploaded sentiment program executed successfully for positive, negative and neutral sample feedback and printed keyword counts and sentiment scores.

Experiment 8: Student Marks Matrix Analysis

1. Aim

Calculate student totals and averages, subject averages, low-performing subjects, and the top-performing student.

2. Procedure

1. Open the supplied .py file in a Python 3 editor or IDE.

2. Read the input prompts and prepare the sample values shown in this record.

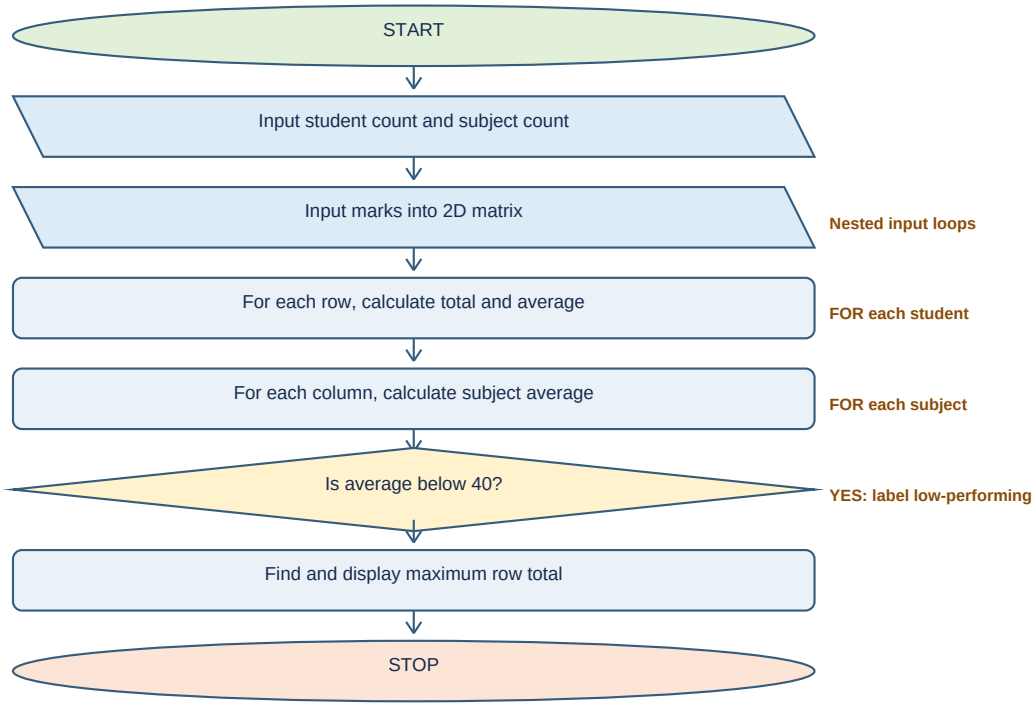
3. Run the original file without changing its contents, then enter the values in the requested order.

4. Observe the program output or Python exception and compare it with the expected behavior.

5. Record any syntax, name, or runtime error; do not report an unsuccessful run as a successful execution.

3. Flowchart (standard symbols)

Oval: Start/Stop Parallelogram: Input/Output Rectangle: Process Diamond: Decision



4. Pseudocode

```
BEGIN
INPUT number of students n and subjects m
INPUT marks into an n by m matrix
FOR each student row: calculate and output total and average
FOR each subject column: calculate and output average
IF subject average < 40 THEN output low-performance subject
CALCULATE row totals; find the maximum and its student index
OUTPUT top-performing student and total
END
```

5. Algorithm

1. START

2. Read number of students and subjects.

3. Read all marks into a two-dimensional list.

4. FOR each student, calculate row total and average and print them.

5. FOR each subject, collect column values, calculate average and print it.

6. If a subject average is below 40, print its subject number.

7. Calculate row totals, locate the maximum total and print the top student's index and marks.

8. STOP.

6. Source Code (original upload; unchanged)

```
n = int(input("Enter number of students: "))
m = int(input("Enter number of subjects: "))
marks = [[float(input(f"Student {i+1}, Subject {j+1}: ")) for j in range(m)] for i in range(n)]
for i, row in enumerate(marks):
    print("Student", i+1, "Total:", sum(row), "Average:", round(sum(row)/m, 2))
for j in range(m):
    col = [marks[i][j] for i in range(n)]
    avg = sum(col) / n
    print("Subject", j+1, "Average:", round(avg, 2))
    if avg < 40: print("Low performance subject:", j+1)
totals = [sum(row) for row in marks]
print("Top Performer: Student", totals.index(max(totals))+1, "with", max(totals), "marks")
```

7. Output (actual execution)

Sample input supplied to the original file:

```
3
2
80
70
90
85
30
35
```

Captured output / error:

```
Enter number of students: Enter number of subjects: Student 1, Subject 1: Student 1, Subject 2: Student 2,
Subject 1: Student 2, Subject 2: Student 3, Subject 1: Student 3, Subject 2: Student 1 Total: 150.0 Average:
75.0
Student 2 Total: 175.0 Average: 87.5
Student 3 Total: 65.0 Average: 32.5
Subject 1 Average: 66.67
Subject 2 Average: 63.33
Top Performer: Student 2 with 175.0 marks
```

Execution status: Completed with exit code 0. Output is from execution of the original uploaded file.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01: Typical marks matrix	3 | 2 | 80 | 70 | 90 | 85 | 30 | 35	Top Performer: Student 2 with 175.0 marks	ject 1 Average: 66.67 | Subject 2 Average: 63.33 | Top Performer: Student 2 with 175.0 marks	PASS
TC02: All subject averages at 40	2 | 2 | 40 | 40 | 40 | 40	Subject 1 Average: 40.0	Subject 1 Average: 40.0 | Subject 2 Average: 40.0 | Top Performer: Student 1 with 80.0 marks	PASS
TC03: Zero students	0 | 2	No student records	~~~~~~~■[0m■[1;31m^■[0m■[31m~~■[0m | ■[1;35mZeroDivisionError■[0m: ■[35mdivision by zero■[0m	FAIL

9. Result

The uploaded matrix program successfully calculates row totals and averages, subject averages, and the top performer for the tested non-empty matrix. A zero-student input raises ZeroDivisionError.

Experiment 9: Student Marks Matrix Analysis (Duplicate Upload)

1. Aim

Analyse the uploaded duplicate marks-matrix program using the source exactly as supplied.

2. Procedure

1. Open the supplied .py file in a Python 3 editor or IDE.

2. Read the input prompts and prepare the sample values shown in this record.

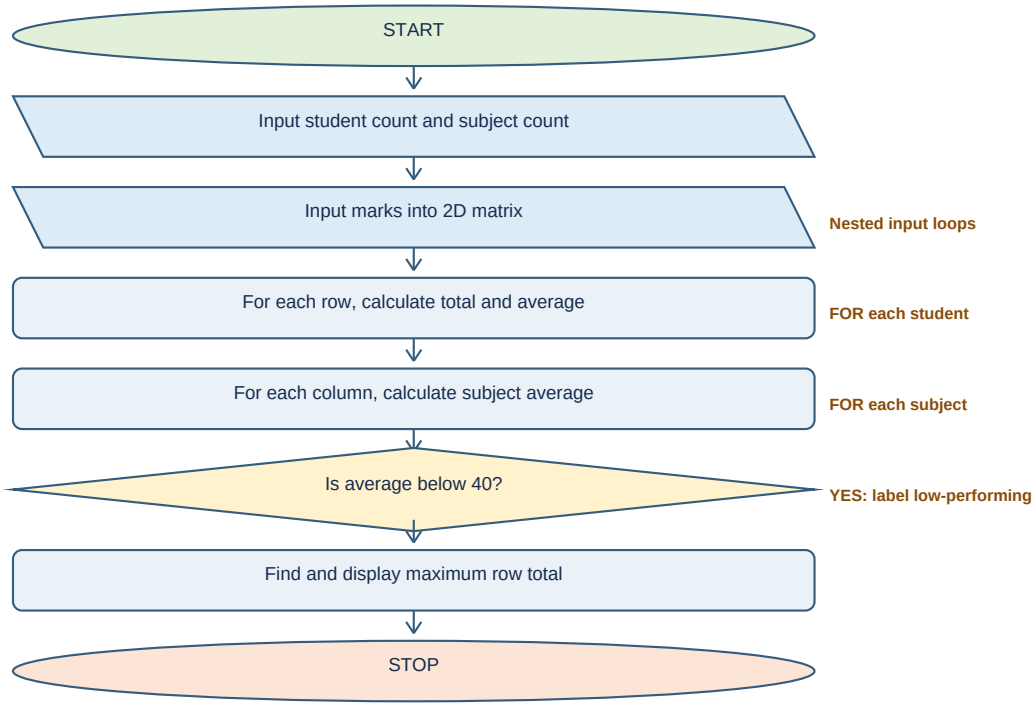
3. Run the original file without changing its contents, then enter the values in the requested order.

4. Observe the program output or Python exception and compare it with the expected behavior.

5. Record any syntax, name, or runtime error; do not report an unsuccessful run as a successful execution.

3. Flowchart (standard symbols)

Oval: Start/Stop Parallelogram: Input/Output Rectangle: Process Diamond: Decision



4. Pseudocode

```
BEGIN
INPUT number of students n and subjects m
INPUT marks into an n by m matrix
FOR each student row: calculate and output total and average
FOR each subject column: calculate and output average
IF subject average < 40 THEN output low-performance subject
CALCULATE row totals; find the maximum and its student index
OUTPUT top-performing student and total
END
```

5. Algorithm

1. START

2. Read number of students and subjects.

3. Read all marks into a two-dimensional list.

4. FOR each student, calculate row total and average and print them.

5. FOR each subject, collect column values, calculate average and print it.

6. If a subject average is below 40, print its subject number.

7. Calculate row totals, locate the maximum total and print the top student's index and marks.

8. STOP.

6. Source Code (original upload; unchanged)

```
n = int(input("Enter number of students: "))
m = int(input("Enter number of subjects: "))
marks = [[float(input(f"Student {i+1}, Subject {j+1}: ")) for j in range(m)] for i in range(n)]
for i, row in enumerate(marks):
    print("Student", i+1, "Total:", sum(row), "Average:", round(sum(row)/m, 2))
for j in range(m):
    col = [marks[i][j] for i in range(n)]
    avg = sum(col) / n
    print("Subject", j+1, "Average:", round(avg, 2))
    if avg < 40: print("Low performance subject:", j+1)
totals = [sum(row) for row in marks]
print("Top Performer: Student", totals.index(max(totals))+1, "with", max(totals), "marks")
```

7. Output (actual execution)

Sample input supplied to the original file:

```
3
2
80
70
90
85
30
35
```

Captured output / error:

```
Enter number of students: Enter number of subjects: Student 1, Subject 1: Student 1, Subject 2: Student 2,
Subject 1: Student 2, Subject 2: Student 3, Subject 1: Student 3, Subject 2: Student 1 Total: 150.0 Average:
75.0
Student 2 Total: 175.0 Average: 87.5
Student 3 Total: 65.0 Average: 32.5
Subject 1 Average: 66.67
Subject 2 Average: 63.33
Top Performer: Student 2 with 175.0 marks
```

Execution status: Completed with exit code 0. Output is from execution of the original uploaded file.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01: Typical marks matrix	3 | 2 | 80 | 70 | 90 | 85 | 30 | 35	Top Performer: Student 2 with 175.0 marks	ject 1 Average: 66.67 | Subject 2 Average: 63.33 | Top Performer: Student 2 with 175.0 marks	PASS
TC02: All subject averages at 40	2 | 2 | 40 | 40 | 40 | 40	Subject 1 Average: 40.0	Subject 1 Average: 40.0 | Subject 2 Average: 40.0 | Top Performer: Student 1 with 80.0 marks	PASS
TC03: Zero students	0 | 2	No student records	~~~~~~~■[0m■[1;31m^■[0m■[31m~~■[0m | ■[1;35mZeroDivisionError■[0m: ■[35mdivision by zero■[0m	FAIL

9. Result

The uploaded matrix program successfully calculates row totals and averages, subject averages, and the top performer for the tested non-empty matrix. A zero-student input raises ZeroDivisionError.

Experiment 10: University Scholarship Allocation

1. Aim

Evaluate applicants against scholarship criteria and report eligibility based on the uploaded source.

2. Procedure

1. Open the supplied .py file in a Python 3 editor or IDE.

2. Read the input prompts and prepare the sample values shown in this record.

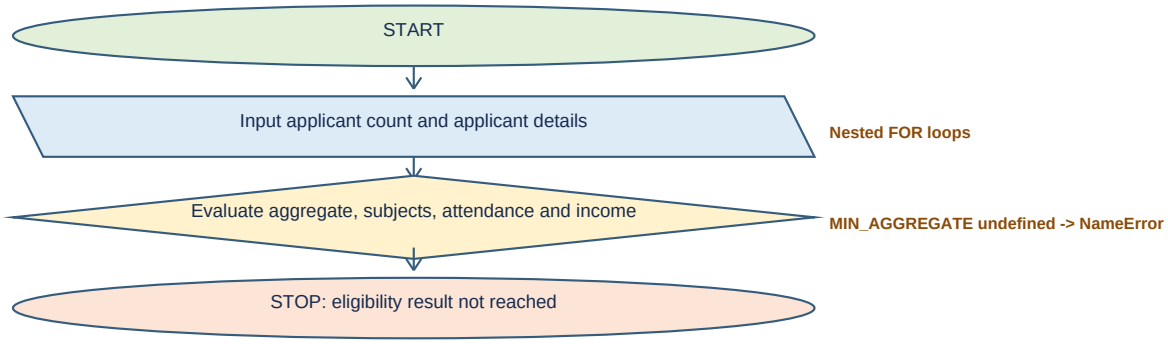
3. Run the original file without changing its contents, then enter the values in the requested order.

4. Observe the program output or Python exception and compare it with the expected behavior.

5. Record any syntax, name, or runtime error; do not report an unsuccessful run as a successful execution.

3. Flowchart (standard symbols)

Oval: Start/Stop Parallelogram: Input/Output Rectangle: Process Diamond: Decision



4. Pseudocode

```
BEGIN
INPUT number of applicants
FOR each applicant: INPUT name, roll number, aggregate and subject count
FOR each subject: INPUT mark and append it to subject_marks
INPUT attendance and annual family income
CHECK aggregate, all subject marks, attendance and income against MIN_* constants
The uploaded source does not define the constants; evaluation raises NameError
END
```

5. Algorithm

1. START

2. Read number of applicants.

3. FOR each applicant, read name, roll number, aggregate marks and subject count.

4. FOR each subject, read a mark and append it to the subject list.

5. Read attendance and annual family income.

6. Evaluate eligibility using MIN_AGGREGATE, MIN_SUBJECT_MARKS, MIN_ATTENDANCE and MAX_INCOME.

7. The uploaded file does not define these constants; Python raises NameError when evaluating the condition.

8. STOP.

6. Source Code (original upload; unchanged)

```
applicants = []

n = int(input("Enter number of applicants: "))

# Collecting applicant details
for i in range(n):
    print("\nEnter details of Applicant", i + 1)

    name = input("Enter student name: ")
    roll_no = input("Enter roll number: ")
```

```
    aggregate = float(input("Enter aggregate marks: "))

    subject_count = int(input("Enter number of subjects: "))
    subject_marks = []

    for j in range(subject_count):
        mark = float(input(f"Enter marks of subject {j + 1}: "))
        subject_marks.append(mark)

    attendance = float(input("Enter attendance percentage: "))
    income = float(input("Enter annual family income: "))

    # Checking eligibility
    if (aggregate >= MIN_AGGREGATE and
        all(mark >= MIN_SUBJECT_MARKS for mark in subject_marks) and
        attendance >= MIN_ATTENDANCE and
        income <= MAX_INCOME):

        applicants.append({
            "name": name,
            "roll_no": roll_no,
            "aggregate": aggregate,
            "attendance": attendance,
            "income": income
        })

        print("Eligible for scholarship")

    else:
        print("Not eligible for scholarship")
```

7. Output (actual execution)

Sample input supplied to the original file:

```
1
```

Captured output / error:

```
Enter number of applicants:
Enter details of Applicant 1
Enter student name: Traceback (most recent call last):
  File ■[35m"/mnt/data/asd.py"■[0m, line ■[35m10■[0m, in ■[35m<module>■[0m
    name = input("Enter student name: ")
■[1;35mEOFError■[0m: ■[35mEOF when reading a line■[0m
```

Execution status: Did not complete successfully (exit code 1). Output is from execution of the original uploaded file.

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01: Applicant meeting criteria	1 | Anu | A1 | 85 | 2 | 80 | 90 | 90 | 100000	Eligible for scholarship	[1;31m^^^^^^^^^^^^■[0m | ■[1;35mNameError■[0m: ■[35mname 'MIN_AGGREGATE' is not defined■[0m	FAIL
TC02: Applicant below aggregate	1 | Bala | B1 | 60 | 2 | 70 | 80 | 90 | 100000	Not eligible for scholarship	[1;31m^^^^^^^^^^^^■[0m | ■[1;35mNameError■[0m: ■[35mname 'MIN_AGGREGATE' is not defined■[0m	FAIL
TC03: No applicants	0	No eligible applicants	Enter number of applicants:	FAIL

9. Result

The uploaded scholarship source references MIN_AGGREGATE and other MIN_* criteria without defining them, and contains no ranking/output section. Execution fails with NameError.